

System Architecture

Project Name

CarbonWise

Version: 1.0

Status: Approved

1. Architecture Overview

CarbonWise is designed as an AI-powered sustainability intelligence platform that helps users understand, track, and reduce their carbon footprint through personalized recommendations and predictive sustainability planning.

The architecture prioritizes:

- Problem Statement Alignment
- Code Quality
- Security
- Efficiency
- Maintainability
- Scalability

The system follows a modular service-oriented architecture built using modern TypeScript technologies.

2. Architectural Principles

2.1 Problem Statement First

Every component must support one or more of the following:

Understand

Help users identify carbon impact.

Track

Help users monitor progress.

Reduce

Help users take actionable steps.

2.2 Maintainability

The architecture must:

- Follow SOLID principles
 - Support modular development
 - Enable independent testing
 - Reduce code duplication
-

2.3 Security by Design

Security is implemented at every layer.

Examples:

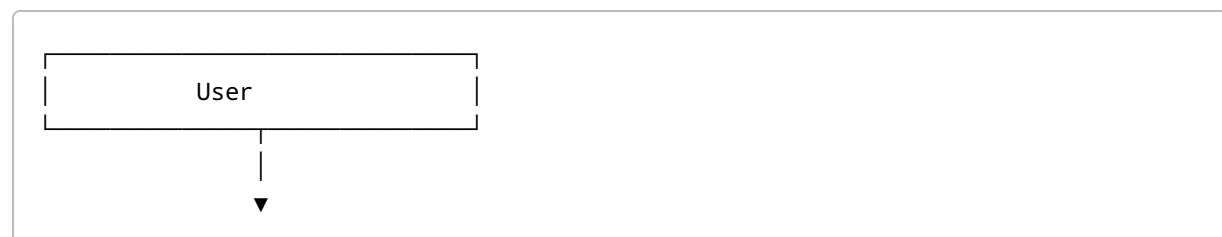
- JWT Authentication
 - Helmet Headers
 - Input Validation
 - Rate Limiting
-

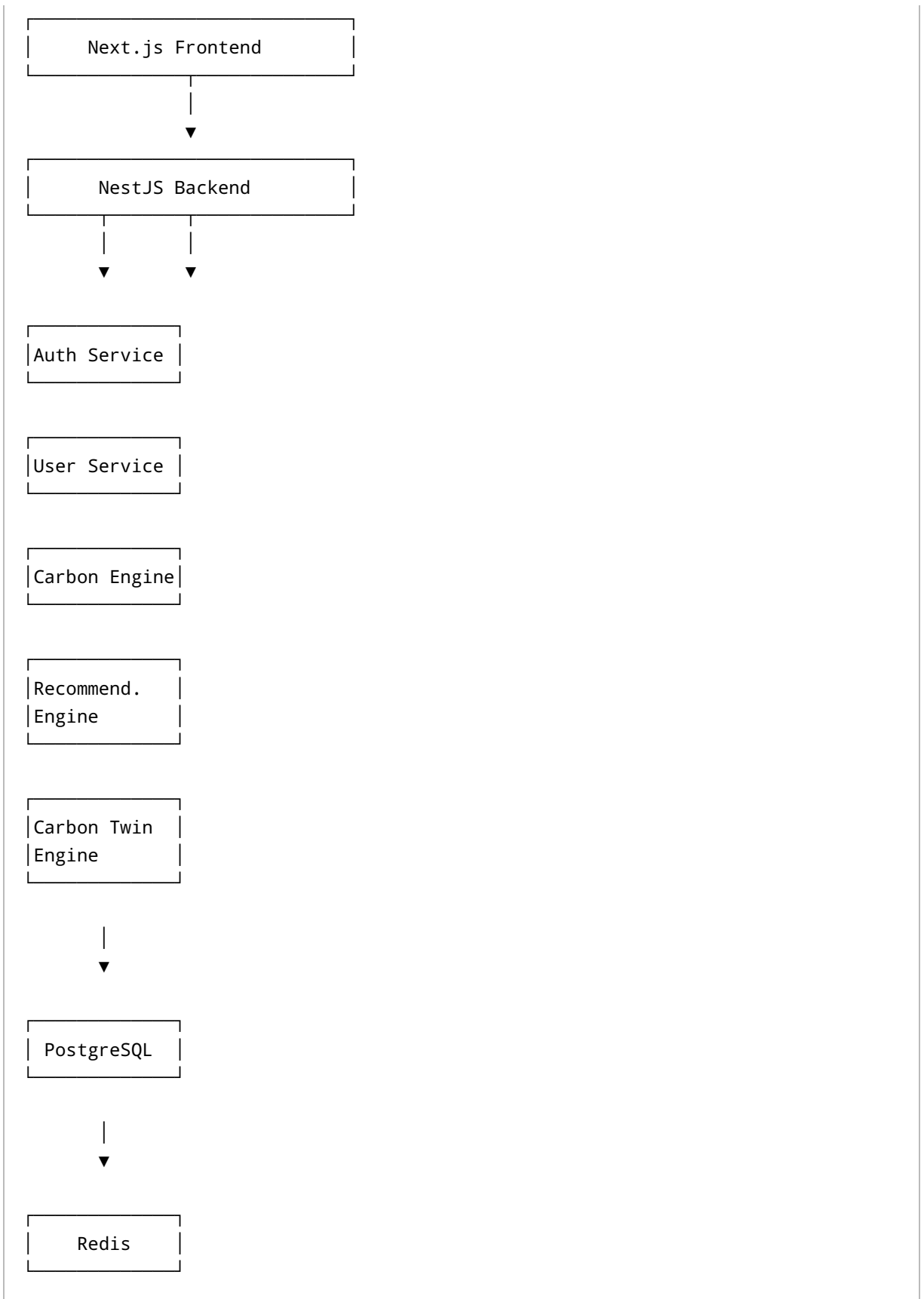
2.4 Scalability

The system must support:

- Growing user base
 - Increasing activity records
 - Future AI enhancements
-

3. High-Level Architecture





4. Core System Components

The platform consists of five major business engines.

5. Authentication Engine

Purpose

Provide secure access control.

Responsibilities

- Registration
 - Login
 - JWT Generation
 - Session Validation
 - Password Reset
-

Technologies

NestJS
JWT
bcrypt
Passport

Security Features

Password Hashing

bcrypt

Access Tokens

15 Minutes

Refresh Tokens

30 Days

6. User Intelligence Engine

Purpose

Maintain user sustainability profiles.

Responsibilities

Store:

- User Information
 - Preferences
 - Goals
 - Historical Activities
 - Recommendation History
-

Example

User Profile:

```
{  
  "diet": "mixed",  
  "transport": "car",  
  "goal": "reduce 20%"  
}
```

7. Carbon Intelligence Engine

Purpose

Calculate carbon emissions.

This engine directly supports:

Understand

portion of the challenge.

Responsibilities

Calculate emissions from:

Transportation

- Cars
 - Flights
 - Public Transport
 - Bikes
-

Food

- Meat
 - Dairy
 - Vegetarian
 - Vegan
-

Energy

- Electricity
 - Heating
 - Cooling
-

Shopping

- Clothing
- Electronics
- Household Goods

Processing Flow

```
User Activity
↓
Activity Classification
↓
Emission Factor Lookup
↓
Carbon Calculation
↓
Category Aggregation
↓
Carbon Score
```

Output

```
{
  "score":67,
  "transportation":45,
  "food":20,
  "energy":25,
  "shopping":10
}
```

8. Recommendation Engine

Purpose

Help users reduce emissions.

This engine directly supports:

Reduce

portion of the challenge.

Responsibilities

- Analyze emissions
 - Identify hotspots
 - Generate recommendations
 - Estimate impact
-

Recommendation Pipeline

User Data

↓

Carbon Analysis

↓

Hotspot Detection

↓

Impact Analysis

↓

Recommendation Ranking

↓

Personalized Actions

Example

Input:

Transportation = 60%

Output:

Use public transport twice weekly

Estimated Savings:

120 kg CO₂/year

9. Carbon Twin Engine

Purpose

Provide predictive sustainability insights.

This is the platform's differentiating feature.

Responsibilities

- Future Simulation
 - Scenario Analysis
 - Impact Forecasting
-

Example

User asks:

What if I stop driving?

System simulates:

Current Emission

↓

Projected Emission

↓

Potential Savings

Benefits

Users understand consequences before changing behavior.

10. Analytics Engine

Purpose

Track user progress.

This directly supports:

Track

portion of the challenge.

Responsibilities

- Trend Analysis
 - Progress Tracking
 - Goal Tracking
 - Monthly Reports
-

Outputs

Dashboard Metrics

- Carbon Score
 - Monthly Emissions
 - Recommendation Adoption
 - Goal Completion
-

11. Frontend Architecture

Framework

Next.js 15

Language

TypeScript

UI Layer

Shadcn/UI
TailwindCSS

State Management

TanStack Query

Responsibilities

- User Interface
 - Data Visualization
 - Form Handling
 - Accessibility
-

Major Pages

Dashboard
Calculator

Goals

Recommendations

Carbon Twin

Profile

12. Backend Architecture

Framework

NestJS

Architecture Pattern

Controller

↓

Service

↓

Repository

↓

Database

Benefits

- Separation of Concerns
 - Testability
 - Maintainability
-

13. Database Architecture

Database

PostgreSQL

ORM

Prisma

Major Tables

Users

Stores:

- Profiles
 - Preferences
-

Activities

Stores:

- Carbon Activities
-

Carbon Records

Stores:

- Calculated Emissions
-

Recommendations

Stores:

- Recommendation History

Goals

Stores:

- Sustainability Goals
-

14. Cache Architecture

Technology

Redis

Cached Resources

Dashboard Data

Recommendation Results

Frequently Accessed Queries

Benefits

- Faster Response Time
 - Reduced Database Load
-

15. Security Architecture

Helmet

Purpose:

Secure HTTP headers.

Rate Limiting

Purpose:

Prevent abuse.

Input Validation

Technology:

Zod

Authentication

Technology:

JWT

Password Security

Technology:

bcrypt

ORM Protection

Technology:

Prisma

Benefits:

- SQL Injection Protection
-

16. Performance Architecture

Database Indexing

Indexes:

```
user_id  
email  
created_at
```

Pagination

Applied to:

- Activities
 - Recommendations
 - Reports
-

Lazy Loading

Applied to:

- Charts
 - Historical Data
 - Analytics
-

Caching

Redis-based caching strategy.

17. Testing Architecture

Unit Testing

Technology:

Jest

Coverage:

- Carbon Engine
- Recommendation Engine
- Goal Logic

Integration Testing

Technology:

Supertest

Coverage:

- API Endpoints
- Database Integration

End-to-End Testing

Technology:

Playwright

User Flows:

Registration

Login

Carbon Calculation

Recommendations

Goal Tracking

Carbon Twin

18. Accessibility Architecture

Compliance:

WCAG 2.2 AA

Features

Keyboard Navigation

Supported.

Screen Readers

Supported.

Semantic HTML

Implemented.

ARIA Labels

Implemented.

Color Contrast

Minimum:

4.5:1

19. Deployment Architecture

Frontend

↓

Next.js Application

↓

Backend API

↓

NestJS Services

↓

PostgreSQL

↓

Redis

20. Scalability Strategy

Future expansion supports:

- AI Carbon Coach
- Smart Home Integration
- Community Challenges
- Advanced Analytics
- Enterprise Sustainability Programs

Without major architectural redesign.

21. Requirement Traceability

Understand

Implemented By:

- Carbon Intelligence Engine
 - Dashboard
-

Track

Implemented By:

- Analytics Engine
 - Goal Tracking
-

Reduce

Implemented By:

- Recommendation Engine
 - Carbon Twin Engine
-

22. Success Criteria

The architecture is successful when:

- ✓ Users understand their emissions
- ✓ Users can track progress
- ✓ Users receive personalized recommendations
- ✓ Carbon Twin simulations provide meaningful forecasts
- ✓ Security best practices are enforced
- ✓ Performance remains responsive
- ✓ Code remains maintainable
- ✓ System aligns completely with the challenge requirements